

CSS

Cascading Style Sheets



CSS: Cascading Style Sheets

CSS es un lenguaje de diseño que **describe la presentación de un documento** escrito en HTML o XML (como XHTML). CSS define cómo se deben mostrar los elementos en la pantalla o papel impreso.

CSS es fundamental en el desarrollo Web ya que **separa el contenido de la presentación**.

CSS **controla el layout de múltiples páginas Web a la vez**, especificar estilos de tipografía, colores, espaciado, tamaño de los elementos, y mucho más. Esto ofrece una gran flexibilidad y control, mejora la accesibilidad del contenido y reduce la complejidad y la repetición en el contenido estructural.



CSS: Cascading Style Sheets (Cont.)

Por “cascading” nos referimos a que una regla de estilo **se aplica en un orden específico**. La cascada de CSS determina qué regla se aplica cuando existen conflictos.

Factores de prioridad:

1. La declaración **!important**
2. Origen del estilo: Puede provenir de los estilos predeterminados del navegador, de los definidos por el autor del sitio o de configuraciones del usuario, por ejemplo, para mejorar la accesibilidad.
3. Especificidad del selector
4. Orden en el código

Importancia de CSS

- ▶ **Separación del contenido y la presentación:** Antes de CSS, el diseño y el contenido estaban entrelazados, lo que significaba que los cambios de estilo requerían modificaciones en el código HTML. Hoy CSS **permite que se separen el contenido (HTML) de la presentación (CSS)**.
- ▶ **Mantenimiento y actualizaciones:** **La separación facilita el mantenimiento y la actualización** de los sitios Web al reducir el tiempo y el esfuerzo requeridos para mantenerlos, especialmente aquellos grandes y complejos.
- ▶ **Accesibilidad:** CSS **asegura que un sitio Web puede ser interpretado correctamente** por lectores de pantalla y otros asistentes tecnológicos utilizados por personas con discapacidades.

Importancia de CSS (Cont.)

- ▶ **Flexibilidad y personalización:** CSS **permite crear diferentes estilos para distintos dispositivos y contextos.**
- ▶ **Rendimiento:** Al tener el contenido separado de la presentación, se **reduce la redundancia y el tamaño de los archivos HTML**, lo que puede mejorar la velocidad de carga de las páginas Web. Los navegadores pueden cachear los archivos CSS acelerando la navegación del usuario por el sitio.
- ▶ **Consistencia:** CSS **asegura una apariencia coherente en todo un sitio Web**, lo que contribuye a una experiencia de usuario consistente.

Evolución de CSS

- ▶ **Pre-CSS (Antes de 1996):** Antes de CSS, el estilo de las páginas Web se controlaba directamente en el HTML. Elementos como `<font>` y atributos como `bgcolor` se utilizaban para agregar estilo, lo que resultaba en código HTML desordenado y difícil de mantener.

CSS1 (1996): La primera versión de CSS, CSS1, fue propuesta por **Håkon Wium Lie** y **Bert Bos** en **1994** y se convirtió en una recomendación del W3C en diciembre de 1996. Introdujo conceptos básicos como la separación del contenido y la presentación, permitiendo a los diseñadores controlar la apariencia de las páginas Web de manera más eficiente.

Evolución de CSS (Cont.)

- ▶ **CSS2 (1998):** CSS2, lanzado en 1998, amplió las capacidades de CSS1. CSS2 permitió una mayor precisión en el diseño y la presentación de las páginas Web.
A medida que CSS ganaba popularidad, los navegadores comenzaron a implementar sus características, aunque no siempre de manera uniforme. Esto llevó a problemas de compatibilidad y a la necesidad de hacks y soluciones específicas para navegadores.
- ▶ **CSS3 (2005 en adelante):** CSS3 introdujo la modularización del lenguaje. Actualmente CSS evoluciona mediante módulos independientes sin versiones numeradas formales.

Estructura de la Sintaxis de CSS

Cada regla en CSS sigue la siguiente estructura:

```
selector {  
    propiedad: valor;  
}
```

- ▶ **Declaración:** propiedad + valor. Por ejemplo, `color: red;`
- ▶ **Selector:** Define qué elementos HTML se van a estilizar.
- ▶ **Propiedad:** Característica que se desea cambiar. Por ejemplo, color, tamaño de fuente, etc.
- ▶ **Valor:** Especifica cómo se aplicará la propiedad.
- ▶ **Llaves {}:** Agrupan las reglas de estilo.
- ▶ **Punto y coma ;:** Separa cada declaración.

Estructura de la Sintaxis de CSS (Cont.)

```
<h1>Título</h1>
```

```
h1 {  
  color: blue;  
  font-size: 24px;  
}
```

- ▶ **h1** es el selector (aplica estilos a todos los `<h1>` en la página).
- ▶ **color** y **font-size** son propiedades.
- ▶ **blue** y **24px** son los valores asignados a cada propiedad.

Tipos de selectores en CSS

- ▶ **Selector de etiqueta:** Aplica estilos a todos los elementos del mismo tipo (h1, p, div).
- ▶ **Selector de clase (.):** Aplica estilos a elementos con una clase específica.

```
.error {  
  color: red;  
}
```

- ▶ **Selector de ID (#):** Aplica estilos a un solo elemento con un ID único.

```
#titulo {  
  font-weight: bold;  
}
```

Tipos de selectores en CSS (Cont.)

- ▶ **Selector de grupo:** Aplica un mismo estilo a múltiples elementos.

```
h1, p, span {  
  text-align: center;  
}
```

- ▶ **Selector de atributo:** Permite aplicar estilos a elementos según los atributos del HTML.

```
input[type="text"] {  
  border: 2px solid blue;  
}
```

Tipos de selectores en CSS (Cont.)

Selectores combinados

Permiten aplicar estilos según la **relación entre elementos HTML** dentro del documento. Tipos principales:

- **Descendiente (espacio)**: Selecciona elementos que están dentro de otro elemento, en cualquier nivel.

```
div p {  
  color: blue;  
}
```

```
<div>  
  <p>Este párrafo se verá azul</p>  
</div>
```

Tipos de selectores en CSS (Cont.)

Selectores combinados

- *Hijo directo (>)*: Selecciona solo los hijos directos.

```
div > p {  
  color: red;  
}
```

```
<div>  
  <p>Este se selecciona</p>  
  <section>  
    <p>Este NO se selecciona</p>  
  </section>  
</div>
```

Tipos de selectores en CSS (Cont.)

Selectores combinados

- ▶ *Hermano adyacente (+)*: Selecciona el elemento inmediatamente después de otro.

```
h1 + p {  
  color: green;  
}
```

```
<h1>Título</h1>  
<p>Este párrafo será verde</p>  
<p>Este no</p>
```

Tipos de selectores en CSS (Cont.)

Selectores combinados

- ▶ **Hermanos generales (~):** Selecciona todos los elementos hermanos que aparecen después del elemento indicado. Los elementos deben tener el mismo padre.

```
h1 ~ p {  
  color: purple;  
}
```

```
<div>  
  <h1>Título</h1>  
  <p>Este será púrpura</p>  
  <p>Este también</p>  
</div>
```

Comentarios en CSS

- Para agregar notas dentro del código CSS:

```
/* Este es un comentario */  
h1 {  
  color: gray; /* Comentario dentro de una regla */  
}
```


Uso de !important en CSS

La palabra reservada **!important** se usa para forzar que una regla tenga prioridad sobre otras, incluso si tienen mayor especificidad.

```
p.deshabilitado {  
    color: gray !important;  
}  
  
p {  
    color: blue;  
}
```

¿Cuándo usarlo?

- ▶ Para sobrescribir estilos en frameworks como Bootstrap.
- ▶ Para forzar la aplicación de un estilo específico.

Evitar su uso excesivo, ya que dificulta el mantenimiento del CSS.

Formas de agregar CSS a una página

- ▶ CSS en línea (inline CSS):

```
<p style="color: blue;">Texto en azul</p>
```

- ▶ CSS interno (dentro de la etiqueta <style> en HTML):

```
<style>
h1, p, span {
  text-align: center;
}
</style>
```

- ▶ CSS externo (archivo .css separado, recomendado):

```
<link rel="stylesheet" href="styles.css">
```

Formas de agregar CSS a una página (Cont.)

Si varias reglas afectan al mismo elemento y tienen la misma especificidad, se aplica la siguiente prioridad:

1. CSS en línea (inline)
2. CSS interno (<style>)
3. CSS externo (.css)

Características claves de CSS:

Estilización de texto con CSS

- ▶ **Tipografía (font-family):** CSS permite especificar la familia tipográfica del texto. Se puede especificar una lista de fuentes, donde el navegador utilizará la primera que tenga disponible. Por ejemplo, *font-family: Arial, sans-serif;*.
- ▶ **Peso del texto (font-weight):** Esta propiedad permite definir el grosor del texto. Por ejemplo, *font-weight: bold;* hará que el texto sea en “negrita”. También se puede usar valores numéricos para especificar el peso: *font-weight: 400;* para normal y *font-weight: 700;* para negrita.
- ▶ **Estilo del texto (font-style):** Se refiere a la inclinación del texto. Puede ser texto normal o cursiva. Por ejemplo, *font-style: italic;*.

Características claves de CSS:

Estilización de texto con CSS (Cont.)

- **Tamaño del texto (font-size):** Con CSS se puede definir el tamaño del texto. Por ejemplo, *font-size: 16px;*

Unidad	Significado	Explicación
px (píxeles)	Absoluto	Tamaño fijo, no cambia con el tamaño del contenedor o la configuración del usuario.
pt (puntos)	Absoluto	Usado en impresión (1 pt = 1/72 de pulgada $\approx$ 0.035 cm), no recomendado en pantallas.
em	Relativo	Se basa en el tamaño del texto del elemento padre .
rem	Relativo	Se basa en el tamaño del texto del elemento raíz (html) asegurando consistencia global.

Características claves de CSS:

Estilización de texto con CSS (Cont.)

- ▶ **Altura de línea (line-height):** Permite definir la distancia entre las líneas de texto, mejorando la legibilidad. Por ejemplo, *line-height: 1.5;* aumenta el espacio entre líneas a 1.5 veces el tamaño del texto.
- ▶ **Alineación del texto (text-align):** CSS permite alinear el texto a la izquierda, derecha, centro o justificarlo. Por ejemplo, *text-align: center;*
- ▶ **Decoración del texto (text-decoration):** Se puede añadir decoración al texto como subrayado, línea sobre el texto o tachado. Por ejemplo: *text-decoration: underline;*

Características claves de CSS:

Estilización de texto con CSS (Cont.)

- ▶ **Transformación del texto (text-transform):** Esta propiedad permite transformar el texto a mayúsculas, minúsculas o capitalizar la primera letra de cada palabra. Por ejemplo, `text-transform: uppercase;`.
- ▶ **Espaciado de letras (letter-spacing):** Permite modificar el espacio entre caracteres. Por ejemplo, `letter-spacing: 2px;`.
- ▶ **Espaciado de palabras (word-spacing):** Similar al espaciado de letras, pero aplicado al espacio entre palabras. Por ejemplo, `word-spacing: 1em;`.

Características claves de CSS:

Uso de colores y fondos

Se puede aplicar colores a texto, bordes, fondos y otros elementos utilizando propiedades como **color** (para texto), **border-color** (para bordes) y **background-color** (para fondos).

Los métodos para definir colores:

- ▶ **Nombres predefinidos:** red, blue, green.
- ▶ **Código hexadecimal:** #FF0000 (rojo puro).
- ▶ **RGB:** rgb(255, 0, 0).
- ▶ **RGBA:** rgba(255, 0, 0, 0.5) (con opacidad).
- ▶ **HSL:** hsl(0, 100%, 50%).
- ▶ **HSLA:** hsla(0, 100%, 50%, 0.5) (con opacidad).

Características claves de CSS:

Uso de colores y fondos (Cont.)

- ▶ **RGBA y HSLA** incluyen el canal alfa para controlar la transparencia (0 totalmente transparente, 1 opaco).
- ▶ **HSL** (Hue, Saturation, Lightness) define el color de manera perceptiva:
 - ▶ **Hue (Tono)**: Representa el color en el espectro (0-360°).
 - ▶ **Saturation (Saturación)**: 0% es gris, 100% es color puro.
 - ▶ **Lightness (Luminosidad)**: 0% es negro, 100% es blanco, 50% es el color puro.

Características claves de CSS:

Uso de colores y fondos (Cont.)

- ▶ **Color de fondo:** La propiedad **background-color** permite establecer un color de fondo para un elemento. Por ejemplo, *background-color: #FFA07A;* (color salmón).
- ▶ **Imágenes de fondo:** Con la propiedad **background-image**, se puede establecer una imagen como fondo de un elemento. Por ejemplo, *background-image: url('imagen.jpg');*
- ▶ **Control de la imagen de fondo:** CSS ofrece varias propiedades para controlar cómo se muestra una imagen de fondo, como **background-repeat** (para controlar la repetición de la imagen), **background-position** (para ajustar la posición de la imagen) y **background-size** (para escalar la imagen de fondo).

Características claves de CSS:

Uso de colores y fondos (Cont.)

- ▶ **Gradientes:** Los gradientes en CSS permiten crear fondos con transiciones suaves entre colores. Existen dos tipos principales:
 - ▶ **Lineales (*Linear gradients*):** Los colores cambian en una sola dirección. Por ejemplo, `background: linear-gradient(to right, red, yellow);`
 - ▶ **Radiales (*Radial gradients*):** Los colores se expanden desde un punto central hacia afuera en forma circular o elíptica. Por ejemplo, `background: radial-gradient(circle, red, yellow);`

Los gradientes permiten transiciones entre colores, permitiendo crear fondos más atractivos sin usar imágenes.

Características claves de CSS:

Modelo de caja, posicionamiento y layouts

El uso de CSS para el posicionamiento y layout es esencial en el diseño Web, ya que define cómo se organizan y visualizan los elementos en la página.

- ▶ **Modelo de caja (Box Model):** En CSS, cada elemento se considera como una caja rectangular. Este modelo de caja incluye el contenido, el padding (relleno), el borde (border) y el margen (margin).
 - ▶ **Contenido (content):** Es la zona donde se muestra el texto o las imágenes del elemento.
 - ▶ **Relleno (padding):** Espacio entre el contenido y el borde. Aumenta el área interna del elemento, pero no su contenido.

Características claves de CSS:

Modelo de caja, posicionamiento y layouts

- ▶ **Borde (*border*)**: Encierra el contenido y el padding. Puede ser de diferentes estilos, anchos y colores.
- ▶ **Margen (*margin*)**: Espacio exterior que separa un elemento de otros. No es parte de la caja visible, pero afecta el espacio alrededor de ella.
- ▶ **Tamaño de caja (*box-sizing*)**: Controla cómo se calcula el tamaño total de un elemento.
 - ▶ ***content-box* (valor por defecto)**: width y height solo incluyen el contenido.
 - ▶ ***border-box***: width y height incluyen contenido, padding y border.

Características claves de CSS:

Modelo de caja, posicionamiento y layouts (Cont.)

- ▶ **Posicionamiento:** CSS ofrece varias propiedades para controlar el posicionamiento de los elementos, como position, top, right, bottom, y left.
 - ▶ **Static:** Es el valor predeterminado. Los elementos se colocan en el orden en que aparecen en el documento.
 - ▶ **Relative:** Posiciona un elemento en relación con su posición original sin alterar el layout del documento.
 - ▶ **Absolute:** Extrae el elemento del flujo normal del documento y lo posiciona en relación con su contenedor más cercano posicionado.

Características claves de CSS: *Modelo de caja, posicionamiento y layouts (Cont.)*

- ▶ **Fixed:** Similar al absolute, pero el elemento se posiciona en relación con la ventana del navegador, permaneciendo fijo al hacer scroll.
- ▶ **Sticky:** Una mezcla entre relative y fixed. El elemento se comporta como relativamente posicionado hasta que su bloque de contenedor cruza un umbral definido, momento en el cual se comporta como fijo.

Características claves de CSS: *Modelo de caja, posicionamiento y layouts (Cont.)*

► Layout (Diseño de página):

- **Display:** Esta propiedad determina cómo se muestra un elemento. Los valores comunes incluyen block, inline, inline-block y none.
- **Flexbox:** Un método de layout que permite organizar los elementos de manera predecible en un contenedor, incluso cuando sus tamaños son desconocidos. Es ideal para componentes de UI y diseños pequeños a medianos.
- **Grid:** Proporciona un sistema de cuadrícula bidimensional para el diseño de páginas, permitiendo posicionar elementos en columnas y filas. Es muy útil para diseños complejos y de gran escala.

Características claves de CSS:

Modelo de caja, posicionamiento y layouts (Cont.)

- ▶ **Columnas y filas (*columns and rows*):** Se utiliza propiedades como *column-count* para textos o *grid-template-columns* y *grid-template-rows* en grid para definir cómo se distribuyen los elementos en columnas y filas.

Buenas prácticas en CSS

- ▶ Usar clases en lugar de IDs para **reutilización**.
- ▶ Usar nombres de clases descriptivos, por ejemplo, menu-principal, boton-primario, etc.
- ▶ Mantener los archivos CSS organizados.
- ▶ Usar CSS externos para mejor mantenimiento.
- ▶ Evitar el uso excesivo de **!important**.

Introducción a CSS3

CSS3 es una evolución significativa respecto a sus predecesores, ofreciendo una gama de nuevas características que permiten crear **páginas Web más ricas y dinámicas**.

CSS3 **se divide en módulos**, lo que permite que las especificaciones se actualicen de manera independiente. Cada módulo aborda diferentes aspectos del diseño y la presentación de páginas Web.



Modularización en CSS3

CSS3 se divide en módulos, lo que permite que las especificaciones se actualicen de manera independiente. Algunos módulos destacados son:

- ▶ **Flexbox (Flexible Box Layout):** Manejo avanzado de distribución de elementos.
- ▶ **Grid Layout:** Diseño basado en cuadrículas para estructuras más flexibles.
- ▶ **Multi-column Layout:** Organización de contenido en múltiples columnas.
- ▶ **Transitions y Animations:** Creación de efectos dinámicos sin JavaScript.
- ▶ **2D/3D Transformations:** Modificación visual de elementos en dos o tres dimensiones.

Modularización en CSS3 (Cont.)

- ▶ **Media Queries:** Diseño responsivo adaptable a diferentes dispositivos.

Como ventajas de la modularización podemos nombrar:

- ▶ **Actualizaciones más rápidas:** Se pueden implementar nuevas características sin esperar una especificación completa.
- ▶ **Especificaciones claras:** Cada módulo tiene su propia documentación específica.
- ▶ **Compatibilidad progresiva:** Los navegadores pueden adoptar características individualmente, asegurando una mejor compatibilidad con versiones anteriores.

CSS3: Características fundamentales

- ▶ **Selectores avanzados:** CSS3 introduce nuevos selectores que mejoran la precisión al aplicar estilos, incluyendo:
 - ▶ Selectores de atributos (*`input[type="text"]`*)
 - ▶ Pseudo-clases (*`a:hover`*, *`nth-child()`*)
 - ▶ Pseudo-elementos (*`p::first-letter`*, *`::before`*, *`::after`*)
- ▶ **Colores y opacidad:** CSS3 añade nuevos modelos de color:
 - ▶ RGB y RGBA: Soporta transparencia con valores de 0 a 1.
 - ▶ HSL y HSLA: Representa los colores de manera más intuitiva (tono, saturación y luminosidad).

CSS3: Características fundamentales (Cont.)

► Fondos y bordes:

- Múltiples fondos (*background-image*): Permite usar varias imágenes superpuestas.
- Bordes redondeados (*border-radius*): Facilita la creación de elementos con esquinas curvas.
- Sombras (*box-shadow*, *text-shadow*): Permite agregar efectos visuales sin necesidad de imágenes.

► Tipografía y estilos de texto:

- Sombra en texto (*text-shadow*): Permite agregar efectos de sombra a las letras.
- Uso de fuentes Web (*@font-face*): Amplía las opciones tipográficas más allá de las fuentes estándar.

CSS3: Características fundamentales (Cont.)

- ▶ **Transformaciones 2D y 3D:**
 - ▶ Rotaciones, escalados y desplazamientos (*transform*).
 - ▶ Perspectiva y profundidad (*perspective*).
- ▶ **Transiciones y animaciones:** CSS3 permite crear efectos de transición y animaciones avanzadas sin necesidad de JavaScript:
 - ▶ Transiciones (*transition*): Cambios suaves en las propiedades CSS.
 - ▶ Animaciones (*@keyframes*): Creación de efectos dinámicos más complejos.

CSS3: Características fundamentales (Cont.)

- ▶ **Media Queries y diseño responsivo:** Uno de los avances más importantes de CSS3 es la introducción de Media Queries, que permiten adaptar el diseño a distintos dispositivos:

```
@media (max-width: 600px) {  
    body {  
        background-color: lightgray;  
    }  
}
```

Facilita el **diseño adaptativo**, mejorando la experiencia del usuario en diferentes pantallas.

CSS3: Características fundamentales (Cont.)

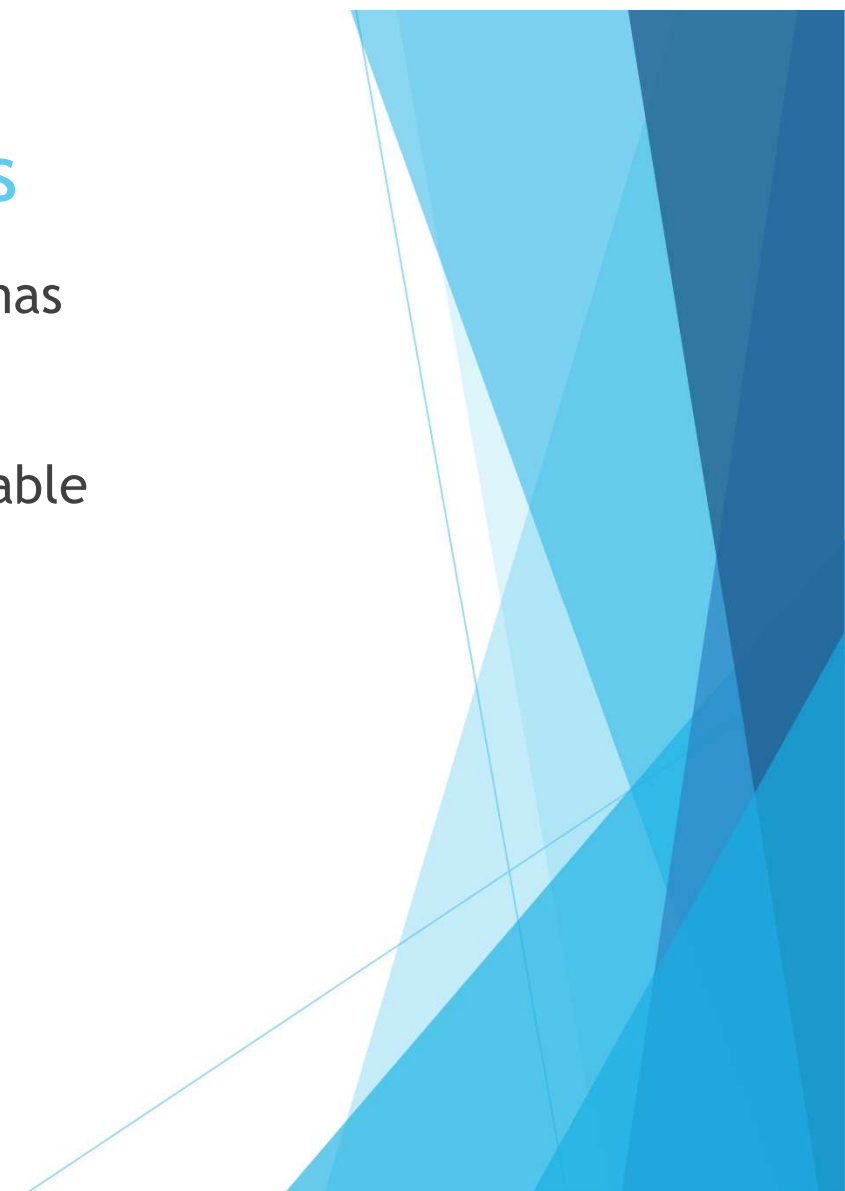
- ▶ **Flexbox y CSS Grid:** CSS3 introduce dos herramientas poderosas para el diseño de interfaces:
 - ▶ **Flexbox:** Distribución eficiente de elementos dentro de un contenedor flexible.
 - ▶ **CSS Grid:** Sistema basado en cuadrículas para crear diseños más complejos.
- ▶ **Variables CSS (Custom Properties):** Permiten definir valores reutilizables como colores, tamaños o fuentes y utilizarlos en diferentes reglas de estilo. Facilitan el mantenimiento y la consistencia del diseño.

Compatibilidad con navegadores

No todos los navegadores implementan las mismas características de CSS al mismo tiempo.

Antes de usar propiedades nuevas es recomendable verificar su soporte en diferentes navegadores.

Herramienta útil: <https://caniuse.com>



Estrategias para manejar la compatibilidad

- ▶ **Degradación agradable (Graceful Degradation):** Diseñar primero para navegadores modernos y luego adaptar el sitio para que también funcione en navegadores más antiguos, aunque con menos funcionalidades.
moderno → antiguo
- ▶ **Mejora progresiva (Progressive Enhancement):** Crear una base funcional para todos los navegadores y luego añadir mejoras que aprovechen las capacidades de los navegadores más modernos.
antiguo → moderno

Comparación entre CSS y CSS3

Característica	CSS (CSS1/CSS2)	CSS3
Estructura	Monolítica, todas las propiedades y capacidades están en un solo nivel.	Modular, dividida en varios módulos que tratan diferentes capacidades.
Selectores	Selectores básicos (ID, clase, tipo, pseudo-clases y pseudo-elementos).	Introduce nuevos selectores como selectores de atributos, de hijo directo, adyacentes y de hermanos generales.
Fondos y bordes	Soporte limitado para colores y bordes.	Permite múltiples imágenes de fondo, bordes redondeados, sombras de caja y bordes con imágenes.
Colores	Soporte básico de colores (nombres y valores hexadecimales).	Introduce RGBA, HSL y HSLA, permitiendo transparencia y más opciones de colores.

Comparación entre CSS y CSS3 (Cont.)

Característica	CSS (CSS1/CSS2)	CSS3
Texto	Capacidades básicas para la manipulación de texto.	Nuevas propiedades como sombra de texto, word-wrap, y la posibilidad de usar fuentes Web con @font-face.
UI	Elementos de interfaz de usuario básicos.	Mejoras en la interfaz de usuario como resize, box-sizing, outline-offset, etc.
Transformaciones	No disponibles.	Transformaciones 2D y 3D, permitiendo rotar, escalar, mover y sesgar elementos.
Transiciones y animaciones	No disponibles.	Permite crear transiciones y animaciones sin necesidad de JavaScript.

Comparación entre CSS y CSS3 (Cont.)

Característica	CSS (CSS1/CSS2)	CSS3
Layout	Diseño basado principalmente en float y posicionamiento.	Introduce Flexbox y Grid, proporcionando nuevas formas de crear layouts complejos de manera más eficiente.
Media Queries	No disponibles.	Permite adaptar el diseño a diferentes tamaños de pantalla y dispositivos, facilitando el diseño responsivo.

Introducción a preprocesadores CSS

Los preprocesadores de CSS **amplían las funcionalidades de CSS** permitiendo usar variables, anidamiento, funciones y reutilización de código.

Algunos preprocesadores populares son:

- ▶ SASS / SCSS
- ▶ LESS

Los navegadores no entienden estos preprocesadores, solo pueden interpretar **CSS estándar**.

Por esta razón, los archivos .sass, .scss, .less **deben convertirse a CSS antes de usarse en una página Web**.

Diferencia entre SASS y SCSS

Ambos son parte de SASS, pero tienen sintaxis diferente:

Característica	SASS	SCSS
Sintaxis	Basada en indentación	Similar a CSS
Uso de llaves {}	No se usa	Se usa
Punto y coma ;	No se usa	Se usa
Facilidad de lectura	Mas limpio pero diferente a CSS	Compatible con CSS estándar

Diferencia entre SASS y SCSS (Cont.)

SCSS

```
$color-principal: blue;  
body {  
    background-color: $color-principal;  
}
```

SASS

```
$color-principal: blue  
body  
    background-color: $color-principal
```

SCSS: Uso de Variables

Las variables permiten almacenar valores reutilizables, evitando la repetición de código, con el objetivo de:

- ▶ Facilitar cambios globales al modificar un solo valor.
- ▶ Mejorar la mantenibilidad del código.

```
$color-principal: blue;
$fuente-principal: Arial, sans-serif;
body {
  background-color: $color-principal;
  font-family: $fuente-principal;
}
```

SCSS: Anidamiento

Permite organizar mejor los estilos, reflejando la estructura del HTML.

```
nav {  
  ul {  
    list-style: none;  
    li {  
      display: inline-block;  
    }  
  }  
}
```

SCSS: Uso de Mixins

Los mixins permiten reutilizar estilos sin repetir código, es decir que:

- ▶ Ayudan a definir estilos reutilizables.
- ▶ Permiten personalización mediante parámetros.

```
@mixin redondeado($radio){  
  border-radius: $radio;  
}  
.caja {  
  @include redondeado(10px);  
}
```

SCSS: Herencia

Permite que un selector herede los estilos de otro. Se usa la palabra reservada *@extend*.

```
.boton {  
  padding: 10px;  
  border: none;  
}  
  
.boton-primario {  
  @extend .boton;  
  background-color: blue;  
}
```

SCSS: Funciones

SASS permite definir funciones personalizadas para manipular valores, con el objetivo de:

- ▶ Reducir el uso de código repetitivo.
- ▶ Permitir cálculos dinámicos dentro de los estilos.

```
@function calcular-espaciado($tamaño) {  
  @return $tamaño * 1.2;  
}  
  
.contenedor {  
  padding: calcular-espaciado(10px);  
}
```

Transpilación de SASS/SCSS

Para que el navegador pueda interpretar los estilos, el código SASS/SCSS debe pasar por un proceso de transpilación que:

- ▶ Traduce el código SASS/SCSS a **CSS estándar**.
- ▶ Elimina las características propias del preprocesador que no forman parte de CSS estándar.
- ▶ Genera un archivo .css que el navegador puede interpretar.

Este proceso suele llamarse "compilación", aunque técnicamente es una transpilación, ya que traduce un lenguaje a otro similar.



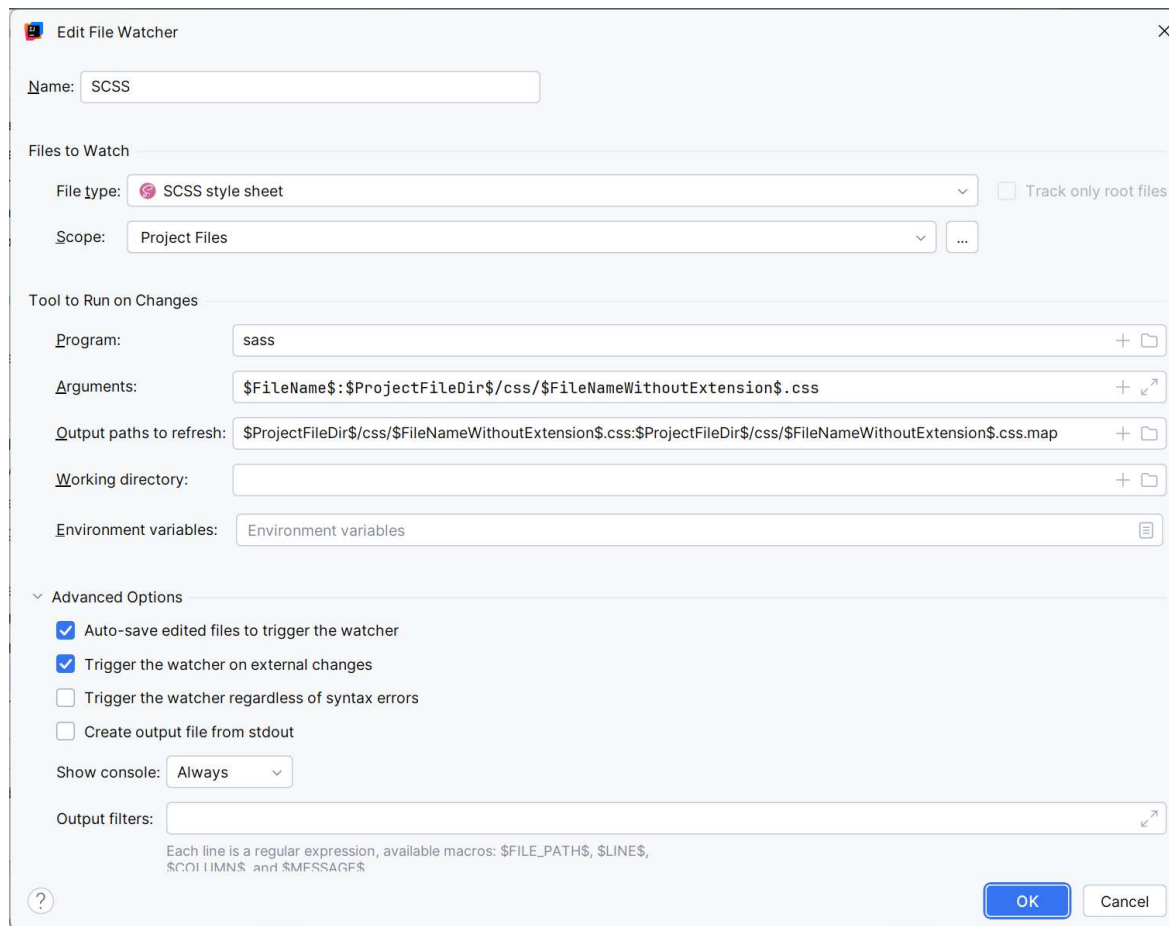
IntelliJ y SASS

- ▶ Instalar en la máquina Node.js: <https://nodejs.org/es>
- ▶ Abrir una consola de Node.js y ejecutar la sentencia:
`# npm install -g sass`
- ▶ Abrir el IDE IntelliJ e instalar el plugin “File watchers”.
- ▶ Ingresar a **File / Settings... / Tools / File Watchers**.
- ▶ Presionar en el icono +.
- ▶ Especificar la siguiente configuración:
 - ▶ Name: **SCSS**
 - ▶ Files to Watch
 - ▶ File type: **SCSS Style Sheet**
 - ▶ Scope: **Project Files**

IntelliJ y SASS (Cont.)

- ▶ Tool to Run on Changes
 - ▶ Program: **sass**
 - ▶ Arguments:
\$FileName\$:\$ProjectFileDir\$/css/\$FileNameWithoutExtension\$.css
 - ▶ Output paths to refresh:
\$ProjectFileDir\$/css/\$FileNameWithoutExtension\$.css:\$ProjectFileDir\$/css/\$FileNameWithoutExtension\$.css.map
- ▶ Advanced Options: Marcar:
 - ▶ Auto-saved edited files to trigger the watcher
 - ▶ Trigger the watcher on external changes
- ▶ Presionar el botón “OK”.

IntelliJ y SASS (Cont.)



The screenshot shows the 'Edit File Watcher' dialog in IntelliJ IDEA. The 'Name' field is set to 'SCSS'. Under 'Files to Watch', the 'File type' is 'SCSS style sheet' and the 'Scope' is 'Project Files'. The 'Tool to Run on Changes' section is configured with the program 'sass', arguments '\$FileName\$:\$ProjectFileDir\$/css/\$FileNameWithoutExtension\$.css', and output paths to refresh '\$ProjectFileDir\$/css/\$FileNameWithoutExtension\$.css:\$ProjectFileDir\$/css/\$FileNameWithoutExtension\$.css.map'. The 'Advanced Options' section has 'Auto-save edited files to trigger the watcher' and 'Trigger the watcher on external changes' checked. The 'Show console' is set to 'Always'. The 'Output filters' field is empty. A note at the bottom states: 'Each line is a regular expression, available macros: \$FILE_PATH\$, \$LINE\$, \$COLUMN\$, and \$MESSAGE\$.'

Edit File Watcher

Name: SCSS

Files to Watch

File type: SCSS style sheet ☐ Track only root files

Scope: Project Files

Tool to Run on Changes

Program: sass

Arguments: \$FileName\$:\$ProjectFileDir\$/css/\$FileNameWithoutExtension\$.css

Output paths to refresh: \$ProjectFileDir\$/css/\$FileNameWithoutExtension\$.css:\$ProjectFileDir\$/css/\$FileNameWithoutExtension\$.css.map

Working directory:

Environment variables: Environment variables

Advanced Options

- ☒ Auto-save edited files to trigger the watcher
- ☒ Trigger the watcher on external changes
- ☐ Trigger the watcher regardless of syntax errors
- ☐ Create output file from stdout

Show console: Always

Output filters:

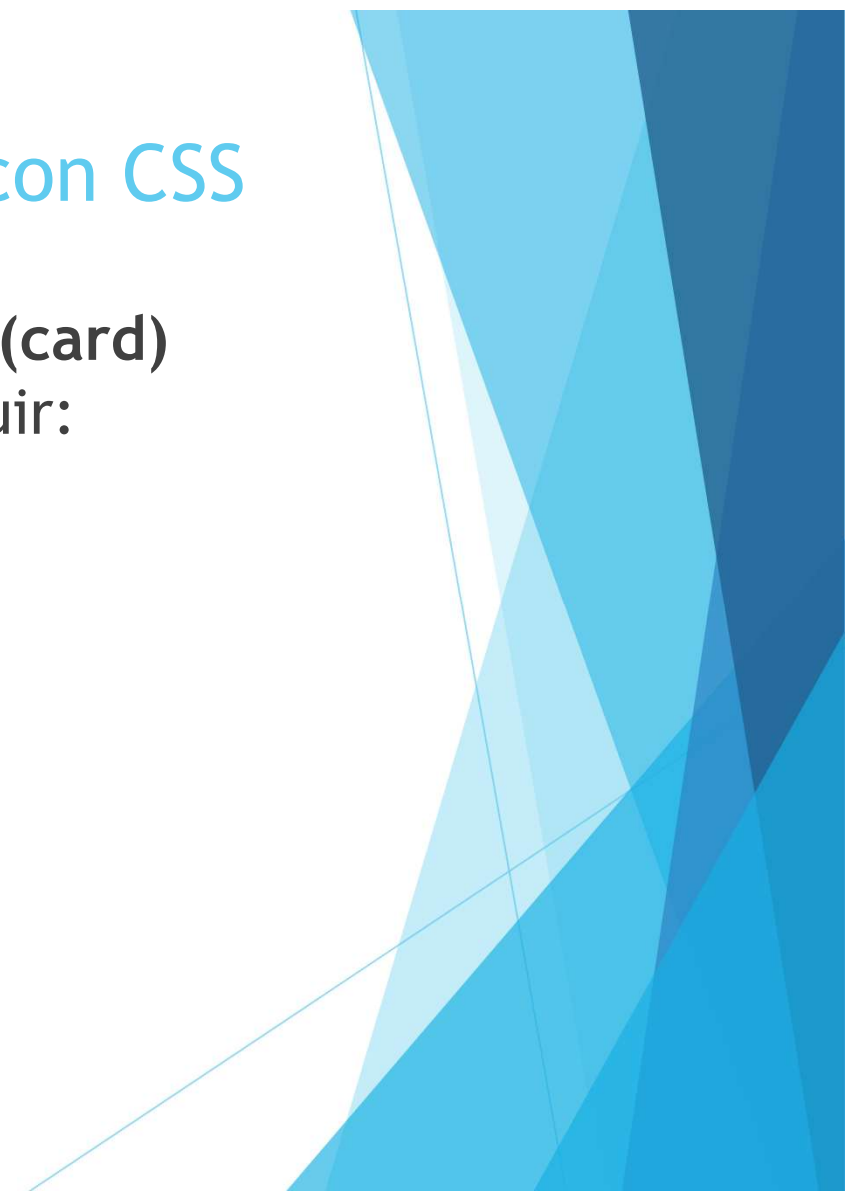
Each line is a regular expression, available macros: \$FILE_PATH\$, \$LINE\$, \$COLUMN\$, and \$MESSAGE\$.

OK Cancel

Prompt 1: Estilizar una tarjeta con CSS

Crea una pequeña **tarjeta de contenido (card)** usando HTML y CSS. La tarjeta debe incluir:

- ▶ Un **título**
- ▶ Un **párrafo**
- ▶ Un **botón**
- ▶ Un **borde redondeado**
- ▶ Un **color principal reutilizado**



Prompt 2: Mejorar con SCSS

Ahora reescribe los estilos usando **SCSS**. Debes:

- ▶ Crear una **variable** para el color principal
- ▶ Crear un **mixin** para el borde redondeado
- ▶ Usar **anidamiento** dentro del estilo que define a la tarjeta



